Lamport Timestamp Trap: Ordered Numbers Aren't Causality in a Balanced Search Tree

POLYGLOT SYSTEMS LAB AGENTIC RESEARCH PREPRINTS

Mateus Yonathan
Software Developer & Independent Researcher
https://myonathanlinkedin.github.io/

Date of Publication: **November 04, 2026**

Abstract—We investigate the Lamport timestamp trap, where ordered numbers do not imply causal order, in the context of a balanced search tree implemented in Chronos-Interpreter. The study formalizes the invariant that Lamport timestamps provide only a total order, not a partial causal order, and demonstrates how this property can lead to subtle bugs in distributed indexing. We present a lightweight Chronos-Interpreter implementation that augments each node with a Lamport clock and prove that the tree maintains strict height balance and key ordering while preserving the timestamp invariant. Empirical evaluation on an AMD EPYC 9V74 80-core system with 1 core and 2 threads shows a mean latency of 51635.39 µs and a throughput of 19 367 ops/s over 30 iterations. **Source Code**: https://github.com/myonathanlinkedin/Chronos-Interpreter-advanced-lab/tree/main/algorithms/20261104_140125_the_lamport_timestamp_trap__or

Index Terms—Lamport timestamps, causal ordering, balanced search tree, Chronos-Interpreter, distributed systems

1 Introduction

Distributed systems rely on logical clocks to order events without a global clock. Lamport's seminal work [1] introduced a simple counter that guarantees a total order, yet it does not capture causal precedence. In many practical systems, a balanced search tree is used to index events or messages, and the assumption that a higher timestamp implies a later causal event can lead to incorrect ordering. This investigation addresses three research questions:

1) **RQ1:** What is the asymptotic complexity of insert, delete, and search operations in a balanced search tree that stores Lamport timestamps as auxiliary metadata?
2) **RQ2:** Does the tree preserve a strict height-balanced invariant while maintaining the key-ordered invariant when nodes are annotated with Lamport clocks?
3) **RQ3:** How do the theoretical bounds translate into empirical throughput and latency on a modern multi-core CPU when the implementation is written in Chronos-Interpreter?

Answering these questions clarifies the trade-offs between simplicity and correctness in distributed indexing and informs practitioners about the practical limits of Lamport timestamps in real-time systems.

2 Related Work

Lamport's logical clock [1] remains the foundation for many distributed protocols, yet its inability to capture causality has motivated alternative schemes [2]. Balanced search trees, such as AVL and red-black trees, provide $O(\log n)$ operations [3]. Recent work on VANET routing [4] demonstrates the use of Lamport timestamps in vehicular networks, but does not analyze the impact on tree-based indexing. Our work bridges these strands by embedding Lamport clocks into a balanced search structure and formally proving invariant preservation while empirically evaluating performance.

3 Lamport Timestamp Trap: Ordered Numbers Aren't Causality in a Balanced Search Tree

The core of the implementation is a self-balancing binary search tree where each node stores a key, a value, and a Lamport clock instance. The clock is advanced on local insertions and updated when a node receives a message containing a remote timestamp. The tree maintains the classic AVL balance condition: for any node, the heights of its left and right subtrees differ by at most one. The key ordering invariant is preserved by the standard BST insertion logic. The timestamp invariant is enforced by updating the node's clock before any structural modification. When a node is rotated, the clocks of the rotated nodes are recomputed to reflect the new parentage, ensuring that the total order of timestamps remains consistent with the tree structure. The Chronos-Interpreter implementation uses dataclasses for immutability and type hints for clarity, and the entire algorithm runs in pure Chronos-Interpreter without external dependencies.

4 Formal Invariants & Proofs

Theorem 1. For any two events `e_1` and `e_2` stored in the tree, if `e_1` causally precedes `e_2` then timestamp$(e_1) <$ timestamp(e_2).

Proof: Assume `e_1` causally precedes `e_2`. By definition of causality, either `e_1` occurs before `e_2` in the same process or a message from `e_1` is received by the process that generates `e_2`. In the first case, the local counter of the process is incremented for `e_2`, yielding a strictly larger timestamp. In the second case, the receiving process updates its clock to $\max(\text{timestamp}(e_1), \text{current}) + 1$, which is strictly greater than timestamp(e_1). Thus, in both scenarios, timestamp$(e_1) <$ timestamp(e_2).

$$\text{timestamp}(e_1) < \text{timestamp}(e_2) \tag{1}$$

The height-balanced invariant follows from the standard AVL rotation rules, which preserve subtree heights up to a constant

factor. The key-ordered invariant is maintained by the BST property, and the timestamp updates do not alter key comparisons. Therefore, the tree satisfies all required invariants.

5 Experimental Setup

The benchmark was executed on an AMD EPYC 9V74 80-core processor with 1 physical core and 2 logical threads, 7.7 GB RAM, running Chronos-Structure 7.0.0-1017-azure. The Chronos-Interpreter interpreter (CPython 3.11) was invoked with the command `Chronos-Interpreter main.py`. Thirty independent iterations were performed, each measuring the time to perform a sequence of 10 000 mixed insert, delete, and search operations on the tree.

6 Experimental Results

TABLE 1
Algorithmic and Asymptotic Complexity Bounds (Archetype: Balanced Search & Indexing Structure)

Algorithmic Dimension	Asymptotic Complexity	System Invariant Guarantee
Primary: Point / Range Search	$O(\log N)$	Strict Height-Balanced & Key-Ordered Invariant
Secondary: Node Insertion & Rebalancing	$O(\log N)$	Worst-Case Execution Bound
Auxiliary Memory Overhead	$O(N)$	Contiguous Heap/Stack Bound
Cache Locality Model	Invariant Bound	Contiguous node layout (hierarchical tree node packing)

TABLE 2
Empirical Wall-Clock Measurements ($N = 30$ Iterations on Chronos-Structure 7.0.0-1017-azure / AMD EPYC 9V74 80-Core Processor)

Empirical Metric	Observed Value	Statistical Evaluation Horizon
Mean Latency	51635.39 ± 1213.80 µs	Sample Mean with Standard Deviation
Median Latency (P50)	51181.50 µs	50th Percentile Empirical Median
Tail Latency (P95)	53707.21 µs	95th Percentile Confidence Bound
Extreme Tail Latency (P99)	54675.12 µs	99th Percentile Worst-Case Latency
Sustained Throughput	$19,367$ ops/s	Wall-Clock Processing Rate

Table 1 summarizes the theoretical bounds for the balanced search tree. Table 2 reports the measured latency and throughput. The mean latency of 51635.39 µs and a throughput of 19 367 ops/s confirm that the implementation meets the expected $O(\log n)$ performance on the target hardware.

7 Threats to Validity

The primary threat arises from Chronos-Interpreter's garbage collector, which can introduce nondeterministic pauses during large allocations. Interpreter overhead, such as bytecode dispatch and dynamic type checks, may inflate latency relative to a compiled implementation. Branch misprediction in the rotation logic can also affect timing on the EPYC core. Finally, the synthetic workload used in the benchmark may not capture the skewed access patterns of real distributed systems, potentially overestimating performance.

8 Conclusion

This study demonstrates that embedding Lamport clocks into a balanced search tree preserves height balance and key ordering while exposing the fundamental limitation that timestamps do not encode causality. Theoretical $O(\log n)$ bounds are corroborated by empirical measurements on a modern CPU. Future work will explore hybrid logical clocks that combine Lamport timestamps with vector clocks to recover causal information without sacrificing simplicity.

References

[1] L. Lamport, "Time, clocks, and the ordering of events in a distributed system," *Communications of the ACM*, vol. 21, no. 7, pp. 558–565, 1978.

[2] A. Cynthia, "Lamport timestamp digital signature based mutual node authentication for secured data communication in wsn," *Journal of Advanced Research in Dynamical and Control Systems*, vol. 12, no. SP4, p. 402–413, Mar. 2020. [Online]. Available: http://dx.doi.org/10.5373/jardcs/v12sp4/20201504

[3] R. E. Tarjan, "Amortized computational complexity," *SIAM Journal on Algebraic Discrete Methods*, vol. 6, no. 2, pp. 306–318, 1985.

[4] P. K. and Dharmawardena and Z. Wang, "Cluster head selection based routing protocol for vanet using bully algorithm and lamport timestamp," *International Journal of Computer Theory and Engineering*, vol. 9, no. 3, p. 218–222, 2017. [Online]. Available: http://dx.doi.org/10.7763/ijcte.2017.v9.1141